

TRUUTH USER PORTAL HIRING CHALLENGE

Applicant Document Submission Portal

Introduction

At truth, we build digital identity, document verification, and fraud detection systems used by global organisations. We operate as a small, fast moving team where engineers have real ownership and impact.

We are preparing to build a new applicant facing User Portal for secure document submission as part of onboarding and verification workflows. This challenge asks you to build a simplified but realistic version of that upcoming portal. The goal is to identify engineers who can deliver clean user experiences, solid backend design, and production minded implementations.

The solution that demonstrates the best balance of usability, engineering quality, and thoughtful design will stand out.

Technology Requirements

- The frontend must be built using React with Typescript. Any styling approach is allowed.
- A Backend for Frontend layer must be implemented so that no truth credentials or API keys are ever exposed in browser code. The backend must be written in Typescript or Python. Node.js with Express, Fastify, or similar frameworks is acceptable.
- The backend will be deployed on Vercel using API routes or serverless functions.
- Bonus points will be awarded if your backend is structured to follow serverless patterns and includes a serverless.yml definition that would allow deployment to AWS Lambda or similar serverless platforms.
- All truth credentials must be stored in backend environment variables. The frontend must never call truth APIs directly.
- A serverless.yml file must be included in the backend project defining service name, runtime, and HTTP endpoints. It does not need to be used for deployment. It is included to assess serverless familiarity.
- Frontend and backend must be deployed to Vercel. Backend may use Vercel serverless functions or API routes. The final submission must be a public URL we can access.

- Persistent storage is required for this challenge to track users, document submissions, and verification results. Any database is acceptable.

The Product You Are Building

You are building a Applicant Document Submission Portal.

Users log in with a simple username and password that you define.

Once logged in, the user is taken to a Document Upload experience. The system must remember previously uploaded documents. When the user logs out and logs back in, the portal must show which documents have been uploaded, which are still processing, and which have completed verification.

Required Documents

Each user must submit three mandatory documents:

1. Australian Passport
2. Australian Driver Licence
3. Resume

The UI must clearly show how many required documents have been uploaded. For example, “1 of 3 required documents uploaded”. A clean, world-class, intuitive design here will score highly.

Each document type must have its own visual container showing the document name, current state, and available actions. The layout and design are up to you. The better the usability and visual design, the higher the score.

A document is considered uploaded once it has been submitted for verification.

Conditional Classification Requirement

When uploading documents:

Australian Passport and Australian Driver Licence must be sent to the truth Classifier API before submission. If classification confirms the document is not an Australian passport or not an Australian driver licence, show a friendly error and block submission.

Resume uploads must skip the classifier step.

Document Submission and Verification Flow

When a user uploads a document:

The file is sent to the backend.

- If required, the backend calls the Classifier API.
- If classification passes, the backend submits the document to the truth Verify API.
- The Verify API returns a documentVerifyId.
- This documentVerifyId must be stored in the database against that user and document type. After submission, the frontend must show that the document has been uploaded and display a processing state for that document.

Document Upload and Result Viewing Behaviour

The UI must clearly show how many required documents have been uploaded. For example, “1 of 3 required documents uploaded”. A clean and intuitive design here will score highly.

After a document is successfully submitted, the frontend must show that the document has been uploaded. While verification is still in progress, the document must display a clear processing state.

Once polling of the verification result completes successfully, the UI must display a “View Result” action for that document. This action must only appear after the backend has received a final response from the verification result endpoint.

When “View Result” is selected, a pop up or modal must open showing the verification result.

At minimum, the result may be displayed as formatted JSON with the ability to copy or download it. Additional points will be awarded if the result is presented in a more readable and thoughtfully designed UI rather than raw JSON, while still allowing access to the raw data.

Asynchronous Verification and Polling

Verification results are not immediate.

For each submitted document, the backend must:

Store the documentVerifyId.

Poll the truth Verify result endpoint every five seconds until status becomes DONE or FAILED.

Store the final verification result in the database.

The frontend must:

- Show a loading or processing state for documents still in verification.
- Enable “View Result” only after verification completes.
- When a user logs in, the system must:
 - Load all submitted documents for that user.
 - For any document still processing, continue polling automatically.
 - For completed documents, enable “View Result” immediately.

Persistence Model

At minimum, your system must persist:

- Users
- Document types
- Document submissions
- documentVerifyId per submission
- Verification status
- Final verification response JSON

Schema design is up to you if persistence works correctly across sessions.

Backend for Frontend Responsibilities

The backend must handle:

- User authentication

- Document submission endpoints
- Persistence of users and submissions
- Classifier API calls for passport and driver licence
- Verify API submission
- Asynchronous polling for results
- Storage of final verification results
- Protection of all truth credentials from exposure
- The frontend must never call truth APIs directly.

Truth API Integration

- Each uploaded document must be passed to the truth APIs through the backend.
- Credentials will be provided separately. They must be stored as backend environment variables.
- The portal must not permanently store document files. Only submission metadata and verification results are stored.
- Details of the API endpoints, credentials, and sample requests are provided in the API Credentials and Integration Info section at the end of this document.

UI and User Experience Expectations

- The portal should feel simple, modern, and easy to use without instructions.
- The upload experience should be intuitive.
- The summary indicator should update immediately after uploads.
- Each document container should clearly show its state.
- Loading and processing states should be obvious.
- Viewing verification results should feel smooth and clear.
- We will strongly evaluate:
 - Visual design quality
 - Clarity of document state presentation
 - Ease of navigation
 - Quality of result display
 - Overall polish and usability
- The best UI combined with solid engineering will score highest.

Error Handling

- Submitting without selecting a file must show an error.
- Uploading unsupported file types must show a friendly error.
- Classification failures must show a friendly message.
- Backend or network failures must show a non-technical error.
- Unexpected errors must not crash the UI.

Deliverables

- A public Git repository containing:
 - Frontend in React with Typescript
 - Backend BFF in Typescript or Python
 - serverless.yml file
 - README with local setup instructions
- A public Vercel URL where the application is accessible.
- Login Credentials

Evaluation Criteria

We will evaluate:

- Correct end to end user flow
- Persistence across login sessions
- Backend for Frontend implementation
- Correct conditional classifier usage
- Correct Verify submission and polling
- Proper storage of documentVerifyId and results
- UI clarity and visual design quality
- Ease of use and polish
- Error handling
- Code readability and maintainability
- Secure handling of credentials
- serverless.yml structure
- Successful deployment

The submission that best balances working functionality, clean design, and thoughtful user experience will be selected.

Final Note

This challenge reflects a real upcoming requirement at truth. The person who excels here is likely someone who will fit naturally into our small and dynamic team and contribute meaningfully to our product growth.

Build it cleanly. Build it thoughtfully. Build it as if it were going into production.

We look forward to reviewing your submission.

API Credentials and Integration Info

This section contains the information required to integrate with truth APIs. You will receive actual credentials separately. Do not hard code credentials in frontend code. Store them only in backend environment variables.

You must refer to this section wherever API integration is required in your implementation.

API Documentation

Getting started guide

<https://truuth.readme.io/reference/getting-started-with-your-api>

You are encouraged to review the documentation before starting the challenge.

Truth Classifier API

Used only for Australian Passport and Australian Driver Licence uploads.

Endpoint

<https://api.au.truuth.id/document-management/v1/classify>

Readme Reference

<https://truuth.readme.io/reference/classify>

Method

POST

Headers

Authorization: Basic <base64(username:password)>

Content-Type: application/json

Request Body Format

```
{
  "images": [
    {
      "image": "<base64 encoded image>",
      "mimeType": "image/jpeg or image/png"
    }
  ]
}
```

Expected Response Example

```
{
  "country": {
    "code": "AUS",
    "name": "Australia"
  },
  "documentType": {
    "code": "PASSPORT",
    "name": "Passport"
  }
}
```

Classification Rules

Passport upload passes only if:

country.code equals AUS

documentType.code equals PASSPORT

Driver Licence upload passes only if:

country.code equals AUS

documentType.code equals DRIVERS_LICENCE

Resume uploads must not call the classifier.

Truth Verify Submission API

Used to submit uploaded documents for verification.

Endpoint

<https://portal.api.us.truuth.id/verify/document-fraud-check>

Readme Reference

<https://truuth.readme.io/reference/submit>

Method

POST

Headers

Authorization: Basic <base64(username:password)>

Content-Type: multipart/form-data

Form Data Fields

documentImage

batchRefId (optional)

batchRefDescription (optional)

Request Body Example

```
{
  "document": {
    "countryCode": "AUS",
    "documentType": "PASSPORT",
    "image": {
      "content": "",
      "mimeType": "image/jpeg"
    }
  },
  "externalRefId": "Testing.",
  "options": {
    "requiredChecks": [
      {
        "name": "ANNOTATION"
      },
      {
        "name": "C2PA"
      },
      {
        "name": "COMPRESSION_HEATMAP"
      },
      {
        "name": "DEEPFAKE_2"
      },
      {
        "name": "DEEPFAKE_3"
      },
      {
        "name": "DEEPFAKE_4"
      }
    ]
  }
}
```

```
{
  "name": "DEEPFAKE_5"
},
{
  "name": "DEEPFAKE_6"
},
{
  "name": "DEEPFAKE_7"
},
{
  "name": "DEEPFAKE"
},
{
  "name": "EOF_COUNT"
},
{
  "name": "HANDWRITING"
},
{
  "name": "INVOICE_DATE_ANOMALY_CHECK"
},
{
  "name": "INVOICE_TOTAL_ANOMALY_CHECK"
},
{
  "name": "SCREENSHOT"
},
{
  "name": "SOFTWARE_EDITOR"
},
{
  "name": "SOFTWARE_FINGERPRINT"
},
{
  "name": "TIMESTAMP"
},
{
  "name": "VENDOR_MISSING_FIELDS"
},
{
  "name": "VENDOR_VALIDATION"
},
{
  "name": "VISUAL_ANOMALY"
},
{
  "name": "WATERMARK_CHECK"
}
]
```

Initial Response Example

```
{  
  "documentVerifyId": "s3H80VD1S9CyhctLfCZd",  
  "status": "PROCESSING"  
}
```

Credential Handling Rules

Credentials must be stored in backend environment variables.

Frontend code must never contain truth credentials or direct calls to truth APIs.

API Credentials

Tenancy -

- Tenant - Truth Hiring
- Tenant Alias - truthhiring
- Region – au (Aws Region - ap-southeast-2)

API Credential -

- API Key - igWS83bV4LMXcs0PpGdl
- Secret –
d806c9e3d9c32078a3b4e5429ca71daed235cbd66bb9ea88f181f4e2ad6335bb

The tenancy will be terminated after the challenge